HIT140 chat notes (14 Oct 2026)

A condensed record of the questions and fixes from this chat. Code is included; explanations are shortened.

1. Excel: get a number from text like `~$5.2M`

Any Excel version (text in A1):

```
=VALUE(SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(A1,"~",""),"$",""),"M",""))
```

Microsoft 365:

```
=VALUE(REGEXEXTRACT(A1,"[0-9.]+"))
```

2. Excel: mix of M and K (convert to millions)

```
=VALUE(REGEXEXTRACT(A1,"[0-9.]+"))/IF(RIGHT(UPPER(TRIM(A1)),1)="K",1000,1)
```

Full dollar amounts instead:

```
=VALUE(REGEXEXTRACT(A1,"[0-
9.]+"))*IF(RIGHT(UPPER(TRIM(A1)),1)="K",1000,1000000)
```

3. Excel: TEXTBEFORE is case-sensitive

```
=TEXTBEFORE(K21,"k",,1)
=IFERROR(VALUE(TEXTBEFORE(K21,"K",,1))/1000, VALUE(TEXTBEFORE(K21,"M",,1)))
```

4. Power Query: Merge vs Append

- **Append** stacks tables (more rows). Same columns needed.

- **Merge** joins tables side by side on a matching column (more columns). Join types: Left Outer (default), Inner, Full Outer, Left Anti.

5. Power Query: format a column as text

Select the column, then Transform > Data Type > Text. Or in the formula bar:

```
= Table.TransformColumnTypes(Source, {{"ColumnName", type text}})
```

6. Excel: XLOOKUP with SUM

`SUM(...)` inside XLOOKUP returns one total, which is not a valid return range.

Total per match:

```
=SUMIF('48teams_squad'!A:A,F2,'48teams_squad'!J:J)
```

Single matching value:

```
=XLOOKUP(F2,'48teams_squad'!A:A,'48teams_squad'!J:J)
```

7. Python: convert a text Date column to dates

```
python
import pandas as pd
df["Date"] = pd.to_datetime(df["Date"])
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)     # day-first dates
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")   # bad entries become NaT
df["Date"] = pd.to_datetime(df["Date"], format="mixed", dayfirst=True)
```

8. Python: Excel serial numbers to dates

```
python
df_f['DOB'] = pd.to_datetime(df_f['DOB'], unit='D', origin='1899-12-30')
```

9. Python: age in years at a reference date

```
python
reference_date = pd.Timestamp('2026-06-11')  # change to the exact date needed

def calculateAge(dob):
  age = reference_date.year - dob.year
  if (reference_date.month, reference_date.day) < (dob.month, dob.day):
    age -= 1
  return age

df['Age'] = df['Dateofbirth'].apply(calculateAge)
```

10. Error: log transform and "Input X contains infinity"

log(0) gives -inf. Use log1p, or check and drop bad rows:

```
python
import numpy as np
df['col_log'] = np.log1p(df['col'])
df = df.replace([np.inf, -np.inf], np.nan).dropna()
```

11. Linear vs logistic regression

- **Linear regression:** predicts a number. Judge with R2, RMSE, MAE.
- **Logistic regression:** predicts a category or probability. Judge with accuracy, precision, recall, confusion matrix.

12. Error: `'numpy.ndarray' object has no attribute 'iloc'`

Use array slicing when `x` is an array:

```
python
X_train, X_test = x[train_index], x[test_index]
```

13. Error: "mix of multiclass and continuous targets"

A classification metric was used on regression output. Use `r2_score` and

`mean_squared_error` for linear regression, or switch to a classifier.

14. Reporting k-fold validation (linear regression)

```
python
from sklearn.model_selection import KFold, cross_validate
from sklearn.linear_model import LinearRegression

model = LinearRegression()
kf = KFold(n_splits=5, shuffle=True, random_state=42)
results = cross_validate(model, x, y, cv=kf,
  scoring=['r2', 'neg_root_mean_squared_error', 'neg_mean_absolute_error'])

r2 = results['test_r2']
rmse = -results['test_neg_root_mean_squared_error']
mae = -results['test_neg_mean_absolute_error']
print(f"R2 mean = {r2.mean():.3f} (SD {r2.std():.3f})")
print(f"RMSE mean = {rmse.mean():.3f} (SD {rmse.std():.3f})")
print(f"MAE mean = {mae.mean():.3f} (SD {mae.std():.3f})")
```

Report: number of folds, shuffle and random seed, mean and SD of each metric across folds.

15. pandas: combine group-by summaries

```
python
dft = df_f.groupby('Country_ANA').agg(
  CAPS_total=('CAPS', 'sum'),
  GOALS_total=('GOALS', 'sum'),
  CAPS_mean=('CAPS', 'mean'),
  GOALS_mean=('GOALS', 'mean'),
  Age_median=('Age', 'median')
).reset_index()
```

16. pandas: join country-by-position averages

```
python
```

```
df_ff = df_f.groupby(['Country_ANA', 'POS'])
posAvg = df_ff[['CAPS', 'GOALS']].mean().unstack('POS')
posAvg.columns = [f'{stat}_{pos}_average' for stat, pos in posAvg.columns]
posAvg = posAvg.reset_index()
dft = dft.merge(posAvg, on='Country_ANA', how='left')
```

How the list comprehension works: each column is a (stat, position) pair such as ('CAPS', 'FW'). The loop splits each pair into `stat` and `pos`, the f-string builds the name `CAPS_FW_average`, and the square brackets collect all the names into a list.

17. seaborn pairplot: label font size

```
python
import seaborn as sns
sns.set_theme(font_scale=1.5)       # scale all text

g = sns.pairplot(df)
for ax in g.axes.flatten():
    ax.set_xlabel(ax.get_xlabel(), fontsize=14)
    ax.set_ylabel(ax.get_ylabel(), fontsize=14)
    ax.tick_params(axis='both', labelsize=10)
```

18. matplotlib `fontdict`

```
python
font_style = {'fontsize': 14, 'fontweight': 'bold', 'color': 'darkblue'}
ax.set_xlabel('Goals', fontdict=font_style)
```

Works on labels and titles, not on tick numbers (use `tick_params`).